Disclaimer: *Dieses OER-Dokument stammt aus dem Projekt proTirone, das über GitHub freie Unterrichtsmaterialien samt Quellcode offeriert. proTirone-Materialien werden unter den Bedingungen der CC-BY-4.0-Lizenz so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).*

LFCX: py2go = Kleines Python Training zwischendurch

1 py2go:Python für Nebenbei-Selbstlerner in eigenen Mini-Scrum-Projekten

Kleine, aufeinander aufbauende Mini-Python-Aufgaben **[→ ZP:Sheet:2]**

1. Geboten werden aufsteigend numerierte Code-Snippets.
2. Ziel ist es - beginnend mit absolut simplen Sachen - gewissermaßen im Vorübergehen, programmieren zu lernen. (= Denn Python ist der beste Pseudocode, den Sie in Prüfungen verwenden können.)
3. Jeder Snippet stellt im Eingangskommentar eine kleine Aufgabe und beschreibt kurz programmiertechnische Hintergründe dazu.
4. Gelöst ist die Aufgabe, wenn der Ihr ausgeführter Code tut, was er soll - die gleichlautende Datei mit *sol* für *Solution* im Namen enthält eine (sic!) Lösungsvariante.
5. Sie sollen so nicht mehr als 10+x min pro Tag aufwenden müssen, um systematisch und praktisch kodieren zu lernen. (Im Unterricht kann dieser Zeitaufwand auch länger angesetzt werden.)

2 py2go:Voraussetzungen

Das Snippets py2go-00.py ist etwas Besonderes: ein lauffähiges Hello World:

ÜBUNG **CRX:py2go:00**

Bringen Sie das Happy-Coding-Python Script unter VSCODE / Python zum Laufen:

- ☐ Installieren Sie Python3 nach einer der Installationsanleitungen im Netz, z.B.:
  - LNX: s. https://opensource.com/article/22/11/python-vs-code-codium
  - W11: s. https://www.python.org/downloads/
- ☐ Verifizieren Sie Ihre Installation = öffnen Sie
  - unter LNX eine *bash* …

  - unter W11 eine *powershell* …
  - und geben Sie jeweils `python3 --help` ein. Erhalten Sie eine python3-Hilfseite, war Ihre Installation erfolgreich.

☐ Laden Sie von https://github.com/protirone/protirone.lessons/ aus dem Ordner `fachinformatik/lf.cx` die Zip-Datei `cx.py2go-snp.zip` herunter:
☐ Entpacken Sie die Zip-Datei unter Linux(!) in einem Arbeitsordner Ihrer Wahl
☐ Laden Sie die Datei `py2go-00-happy-coding.sh` in Ihren VSCODE-Editor
☐ Öffnen Sie von VSCOD[E|IUM] ein Terminal und geben Sie `python3 py2go-00-happy-coding.py` in Ihre Shell ein. Sie sollten danach ein *einen freundlichen Gruß sehen* sehen.
☐ Öffnen Sie danach Ihre *vscode* bzw. *vscodium* Instanz:

  ☐ Installieren Sie die Extensionen *Python* und *Python-Debugger* von *ms-python*.
  ☐ Öffnen Sie wiederum die Datei *py2go-00*.py* = Laden Sie sie in ein VSCode-Fenster.
  ☐ Klicken Sie rechts oben auf das Dreieck 'Run Python File'. (Variante: Öffnen Sie mit einem Klick auf die rechte Maustaste das Kontextmenue und wählen Sie `Run Python / Runy Python File in Terminal` an). Sie sollten (unten in einem gesonderten Fenster) dieselbe Ausgabe bekommen, wie oben.

Jetzt Sie sind bereit

---

3 py2go als personalisiert Mini-Scrum-Projekte

- Sprint-Definition:
  - 1. Entscheiden Sie sich für einen (kleinen) Timeslot als Ihre heutige Arbeitszeit.
  - 2. Lesen Sie sich 3(+-x) Pythonaufgaben (= die Dateien ohne `.sol.` im Dateinamen) durch.
- Planning-Poker:
  - 3. Schätzen Sie informell die gefühlte Komplexität der Aufgaben. Die Komplexität geben Sie in Storypoints an:
    * 1 `STP` : sehr einfach
    * 2 `STP` : einfach
    * 3 `STP` : mäßig komplex
    * 5 `STP` : komplex
    * 8 `STP` : sehr komplex

    * `13` STP : überaus komplex
    * `20` STP : nicht wiorklich schätzbar
    * ...
  - 4. Entscheiden Sie, wie viele Aufgaben Sie in ihrem Timeslot lösen werden.

- Sprint-Backlog:
  - 5. Verschieben Sie diese Aufgaben in Ihren Sprint-Backlog-Ordner
  - 6. Lösen Sie in Ihrem Timeslot, so viele der angesetzten Aufgaben, wie möglich. Gehen Sie frei vor. Die Reihenfolge ist technisch egal. Allerdings bauen die Aufgaben aufeinander auf.

- Sprint-Review:
  - 7. Verschieben Sie ungelösten Aufgaben ins Scrum-Backlog zurück.
  - 8. Vergleichen Sie (innerhalb ihres Timeslots) bei den Aufgaben, die Sie gelöst haben, Ihre Lösung mit den vorgeschlagenen (= die Dateien mit `.sol.` im Dateinamen). Sehen Sie sich aber keine Lösung an, bevor Sie sie selbst gelöst haben.

- Sprint-Retrospektive:
  - 9. Überlegen Sie, was Sie beim nächsten Sprint besser machen können, um das gesetzte Ziel inhaltlich und von der Anzahl her besser zu erreichen.

ÜBUNG **CRX:py2go:XX**

☐ Arbeiten Sie mit die Aufgaben aus dem snp.py2go-Ordner in Ihrem privaten Scrum-Mode der Reihe nach durch.

Hinweis:

- Die Aufgaben gibt es in einer Version ohne Lösung und mit Lösung. Die Idee ist, die Lösung zum Vergleich mit der selbst programmierten heranzuziehen.
- Weitere Detailinfos finden Sie unter https://www.w3schools.com/python/
